

Report Out

Cayleigh Haag

Summer Tasks

Modification and Improvement of the C17 Playback Tool

Implementation of Jira and Confluence for C17 FDR Hot Start

Navaid and Airfield Spatial Hashing Compression Algorithm



Modification and Improvement of C17 Playback Tool

Version 1.30.0

Merged Versions &
Added Menu Bar

- Combined two parallel versions of playback into one cohesive version
 - Merged Live playback for Alta/DT Enet2 Devices
 - Merged CNC panels
- Added in a MCD5 Panel
- Added in new keyboard options for magnifying CNC1, CNC2, and MCD5
- Added in new keyboard options for Rewind and Fast Forward
- Replaced the button panel with a menu bar containing the following additional features
 - Preset formats
 - Custom formats
 - Panels
 - Magnify

CNC1 Panel



Old Button Panel



New Menu Bar



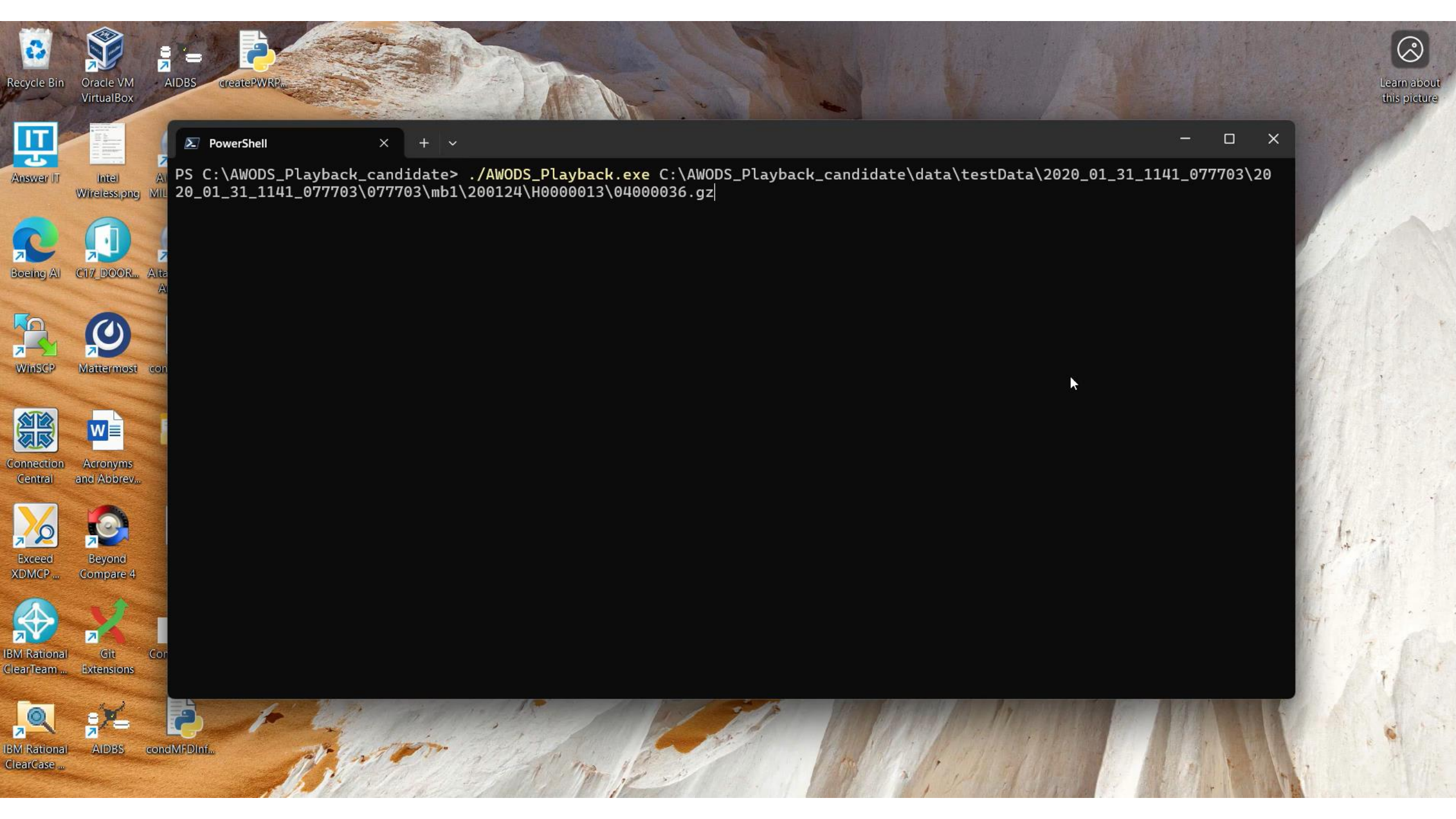
Version 1.30.0

Implemented New and Easier
Way to Select Data

- Simplified the 'DIR' and 'OPEN' commands to 'Playback'
 - Users can select any directory at or above the 'minute' directory to playback their data
 - Users no longer must have a tail number directory
- Added/updated ways to run playback
 - From 'Playback' (see above)
 - From the command line
 - Using optional arguments --ymd and --hms
 - From the .ini file

Examples of correct calls to playback, for the provided sample data:

Command	Result
<code>./playback.exe C:\Gitd\data</code>	Runs first data found
<code>./playback.exe C:\Gitd\data\testData\...\077703</code>	Runs first data found
<code>./playback.exe C:\Gitd\data\testData\...\077703\mb1\200124\H0000013</code>	Runs data on 01/24/2020 at 1 pm for the first minute found
<code>./playback.exe C:\Gitd\data\testData\...\077703\mb1\200124 --hms 130000</code>	Runs data on 01/24/2020 at 1 pm for the first minute found
<code>./playback.exe C:\Gitd\data\testData\...\077703\mb1\200124\H0000013\04000043.gz</code>	Runs data on 01/24/2020 at 1:43 pm
<code>./playback.exe C:\Gitd\data\testData\...\077703\mb1\200124\H0000013 --hms 004300</code>	Runs data on 01/24/2020 at 1:43 pm
<code>./playback.exe C:\Gitd\data\testData\...\077703\mb1\200124 --hms 134300</code>	Runs data on 01/24/2020 at 1:43 pm
<code>./playback.exe C:\Gitd\data\testData\...\077703\mb1 --ymd 200124 --hms 134300</code>	Runs data on 01/24/2020 at 1:43 pm
<code>./playback.exe C:\Gitd\data\testData\...\077703\mb1\200124\H0000013\04000036.gz --hms 134300</code>	Runs data on 01/24/2020 at 1:36 pm (ignoring the optional argument since the path specifies the minute)



Learn about
this picture

```
PowerShell
PS C:\AWODS_Playback_candidate> ./AWODS_Playback.exe C:\AWODS_Playback_candidate\data\testData\2020_01_31_1141_077703\2020_01_31_1141_077703\077703\mb1\200124\H0000013\04000036.gz
```

Version 1.30.0

Introduced Screen Formatting

- Added the ability to turn panels on or off
- Updated the ability to magnify panels
- Added the ability to change screen formats ('Preset Formats')
- Added the ability to create, save, and delete 'Custom Formats' which persist between runs of playback
- Added the ability to reset formats to last used preset or custom format



PLAYBACK 1.30.0 SEP 3 2025 TAIL/MEH 077179
GMT 02/20/20 14:59:02
MSG 0175109/6242304/1022324/744573/5809345



Implementation of Jira for C17 FDR Hot Start

Backlog

Board ▾



QUICK FILTERS:

All Epics

Stories Missing Epic

No Points

Only My Issues

Recently Updated

VERSIONS
EPICS

▼ **FDR Hot Start Sprint 3** 9 issues - **ACTIVE** **AUTO-MANAGED**

22 24 0



01/Sep/25 9:29 AM • 15/Sep/25 9:29 AM [View linked pages](#)

= C17FDR-466 Create a Prototype Health Monitor (HM) Application	Implement Host Arc... 8
= C17FDR-474 Create MC/OFP COM Definitions for 4 New MFD LRUs	Design, Develop, & T... 8
= C17FDR-469 Update the Linux Init Scripts to Start all FDR Software Components	Implement Host Arc... 3
= C17FDR-465 Create a Startup Script for the Compositor (COMP)	Implement Host Arc... 3
= C17FDR-467 Remove the MFD->COMP Startup Dependency	Design, Develop, & T... 3
= C17FDR-399 Update Lynx MOSA.inc	Raptor Lake Integrati... 5
= C17FDR-398 Update QEMU MFD VM to Use Static Libraries	Implement Host Arc... 5
= C17FDR-145 Integrate Raptor Lake Board for CIP or MFD	Raptor Lake Integrati... 8
= C17FDR-400 Integrate Lynx MOSA.ic Prototype Software with Curtiss Wright Loaned Board	Raptor Lake Integrati... 3

Backlog 28 issues -

Create sprint



= C17FDR-124 Understand/resolve unexpectedly long ping times between the I/O Bridge and the MFD SBC	Design, Develop, & T...
= C17FDR-123 Fix background symbology randomly drops and comes back	Design, Develop, & T...
= C17FDR-125 Fix DM when KCIP is disconnected from the bench	Design, Develop, & T...
= C17FDR-126 Disable Hyperthreading at BIOS Level (when ready)	Implement Host Arc...
= C17FDR-127 Fully prove out the "segmented" 1553 bus approach for the MFDs	Implement Host Arc...

PAGE TREE

- FDR Prototype Build Planning & Trac
- Jira Label Management
- ▼ **How-to articles**
 - How To Add a New Team Member:
 - How To Bulk Import Issues into Jira
 - How To Create a Filter for a Board
 - How To Make a Bulk Change
 - How To Set a Default Structure
 - How To Delete Duplicates in Jira
 - How To Set Up Email Encryption fo
- › C17FDR Hot Start Software
- › C17FDR Hot Start Electrical
- › C17FDR Hot Start SRR-SDR
- › C17FDR Hot Start SCFTA
- › C17FDR Hot Start HFE

How-to articles

Created by Haag (US), Cayleigh E, last updated about 5 hours ago • 1 minute read

Add how-to article

Title	Creator	Modified
How To Set Up Email Encryption for Jira	Haag (US), Cayleigh E	about 5 hours ago
How To Bulk Import Issues into Jira from a CSV	Haag (US), Cayleigh E	Sep 05, 2025
How To Set a Default Structure	Haag (US), Cayleigh E	Sep 05, 2025
How To Make a Bulk Change	Haag (US), Cayleigh E	Sep 05, 2025
How To Create a Filter for a Board and Backlog	Haag (US), Cayleigh E	Sep 05, 2025
How To Add a New Team Members to Jira	Haag (US), Cayleigh E	Sep 05, 2025
How To Delete Duplicates in Jira	Haag (US), Cayleigh E	Sep 04, 2025

 Like Be the first to like this

No labels 



Write a comment...

How To Bulk Import Issues into Jira from a CSV

Created by Haag (US), Cayleigh E, last updated on Sep 05, 2025 • 4 minute read

Steps:

1. Populate a copy of this [excel sheet](#) with issues (stories, epics, and/or capabilities) for import
 - a. Each column represents a field for a Jira issue
 - b. Each row represents one instance of a Jira issue that you would like to import

	A	B	C	D	E	F	
1	Summary	Issue Type	Description	Team	Team ID (populated automatically, see table)	Epic Name (only for epics)	Epic Link (only
2	Example Story	Story	Description here	C17FDR Hot Start HFE	522		C17FDR-160
3	Example Epic	Epic	Description here	C17FDR Hot Start SRR-SDR	506	Example Epic	C17FDR-2
4	Example Capability	Capability	Description here	C17FDR Hot Start Electrical	505		
5							

- c. See ***Notes below for instructions on how to fill this spreadsheet out
 - d. **DO NOT MODIFY** the headers of columns or the table contained in columns I through K
2. Save this excel sheet as a 'CSV UTF-8 (Comma delimited)(*.csv)' file

↑ OneDrive - The Boeing Company

Import Bulk Issues in Jira

CSV UTF-8 (Comma delimited) (*.csv)

Unsupported

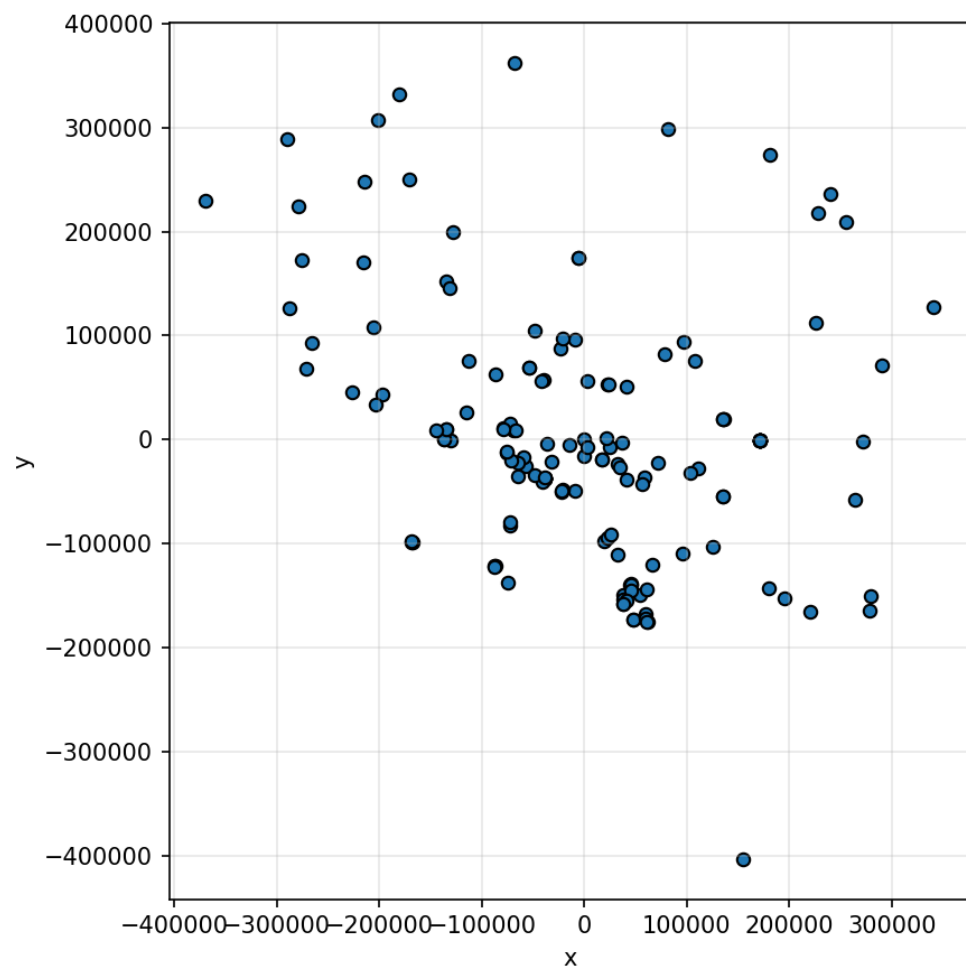
 Save

- a.
3. Import the CSV file in Jira
 - a. In Jira, click the dropdown option titled 'Issues' (at the top of the screen) (**Issues** ▼)
 - b. Click **Import Issues from CSV**
 - c. For 'CSV Source File', choose the excel sheet **that has been saved as a .csv**
 - i. Example: **CSV Source File** Bulk_Import_Issues_To_Jira.csv 
 - d. You can either download a copy of this [pre-set configuration file](#) (prefer this) or set your own configuration (see step 5)
 - i. If you opt to download this file, check the box that says 'Use an existing configuration file' and upload the file (☒ **Use an existing configuration file**)



Navaid and Airfield Spatial Hashing Compression Algorithm

Before Compression



SPATIAL HASH

- Data structure with the following inputs:
 - x_0 & y_0 : the bottom of the grid, relative to input message lat/long coordinates (transformed to flat earth)
 - $x_0 * 2$ represents width of grid
 - $y_0 * 2$ represents height of grid
 - (0,0) is center of grid
 - cell_w and cell_h: the dimensions of a subsection of the grid (a “cell”)
 - Each cell_w x cell_h subsection has a unique index in our hash
 - Cols: **width of grid / cell_w**
 - Rows: **height of grid / cell_h**
 - Buckets: a 2D array of size cols * rows
 - Each ‘bucket’ stores any hashed points at the associated unique index, specified by the appropriate column and row
- Expected $O(1)$ insertion, lookup and deletion
 - Worst case $O(n)$ (can implement tree-based buckets to make lookup and deletion worst-case $O(\log n)$)

GENERAL ALGORITHM FORMAT (1/2)

- Parameters
 - Group_msgs: a section of a DataFrame with messages from a consecutive group of blocks 1 – 12
 - Ids: the associated row indices of these messages
 - Point_size: the length and width you assign to a point (at what distance would you like two points to be condensed)
 - Cell_size: used to determine dimensions (length and width) of a cell in the spatial hash map
- Before while-loop
 - Create a queue called 'Points'
 - For each message in group_msgs add 4 points to this queue:
 - Each of these 4 points have the same id_ (the index passed in through ids)
 - Each point keeps track of what type of point it is (top left, top right, bottom left, bottom right)
 - 1 point for top left corner ($x = \text{center_x} - \text{point_size} / 2$ and $y = \text{center_y} + \text{point_size} / 2$)
 - 1 point for top right corner ($x = \text{center_x} + \text{point_size} / 2$ and $y = \text{center_y} + \text{point_size} / 2$)
 - 1 point for bottom left corner ($x = \text{center_x} - \text{point_size} / 2$ and $y = \text{center_y} - \text{point_size} / 2$)
 - 1 point for bottom right corner ($x = \text{center_x} + \text{point_size} / 2$ and $y = \text{center_y} - \text{point_size} / 2$)

GENERAL ALGORITHM FORMAT (2/2)

- While the queue is not empty

- Remove and store the topmost point off the queue
- Find the location of this point in the hash map using:
 - $i = \text{int}(\text{floor}((x - \text{self.x0}) / \text{self.cell_w}))$ (i in range 0 (inclusive) to self.numCols (exclusive))
 - $j = \text{int}(\text{floor}((y - \text{self.y0}) / \text{self.cell_h}))$ (j in range 0 (inclusive) to self.numRows (exclusive))

^ get the row and column of this point

- Get an index that is unique for this row and column: $\text{idx} = j * \text{self.cols} + i$ (idx in range 0 (inclusive) to self.rows * self.cols (exclusive))
- Get all other points stored at that index (expect constant points)
 - If there are no other points at that index → insert that point into the hash function (no conflicts)
 - Otherwise check for conflicts which occur if:
 - Points do not have the same id_ (are not representative of the same message) AND
 - Points satisfy $\text{hypot}(\text{pointA.x} - \text{pointB.x}, \text{pointA.y} - \text{pointB.y}) < \text{cell_size}$
 - If there is a conflict
 - Combine the two points by finding a new center. Average the two original centers, adjusting for what types of points they are (so re-centering first then averaging centers). Assign this combined point the id of the last point removed from the queue for consistency.
 - Update the DataFrame (zero out pointB and update pointA with information for combined point)
 - Add four new points to 'Points' queue, similar to how points were added initially (top left, top right, bottom left, bottom right)
 - Remove pointA and pointB from the hash (requires checking 8 cells surrounding the index found above and removing if cell with the same id is found)
 - Remove pointA from the queue (requires checking next 3 slots in queue and removing if the same id is found)
 - If we check all points at this index and there are no conflicts → insert that point into the hash function

RUNTIME ANALYSIS

- n is the number of messages in `msg_group`
- Initialize points queue: $O(n)$
- While loop: $O(n)$ iterations initially (increases as points added to queue after conflict but always condensing, 2 points $\rightarrow$ 1 point)
 - Get cell index: $O(1)$
 - Get other points at index: $O(1)$
 - Check for conflicts with other points: expected $O(1)$ using hash
 - Removing points from hash: expected $O(1)$ using hash
 - Removing point from queue: $O(1)$
 - Adding new points to hash: $O(1)$
 - Adding new points to queue (for condensation after conflict): $O(1)$
- Overall $O(n + n) = \textbf{expected } O(n) \textbf{ runtime}$

DEMONSTRATION OF COMPRESSION

Inputs to spatial hash:

BOTTOM_LEFT_X = -400_400

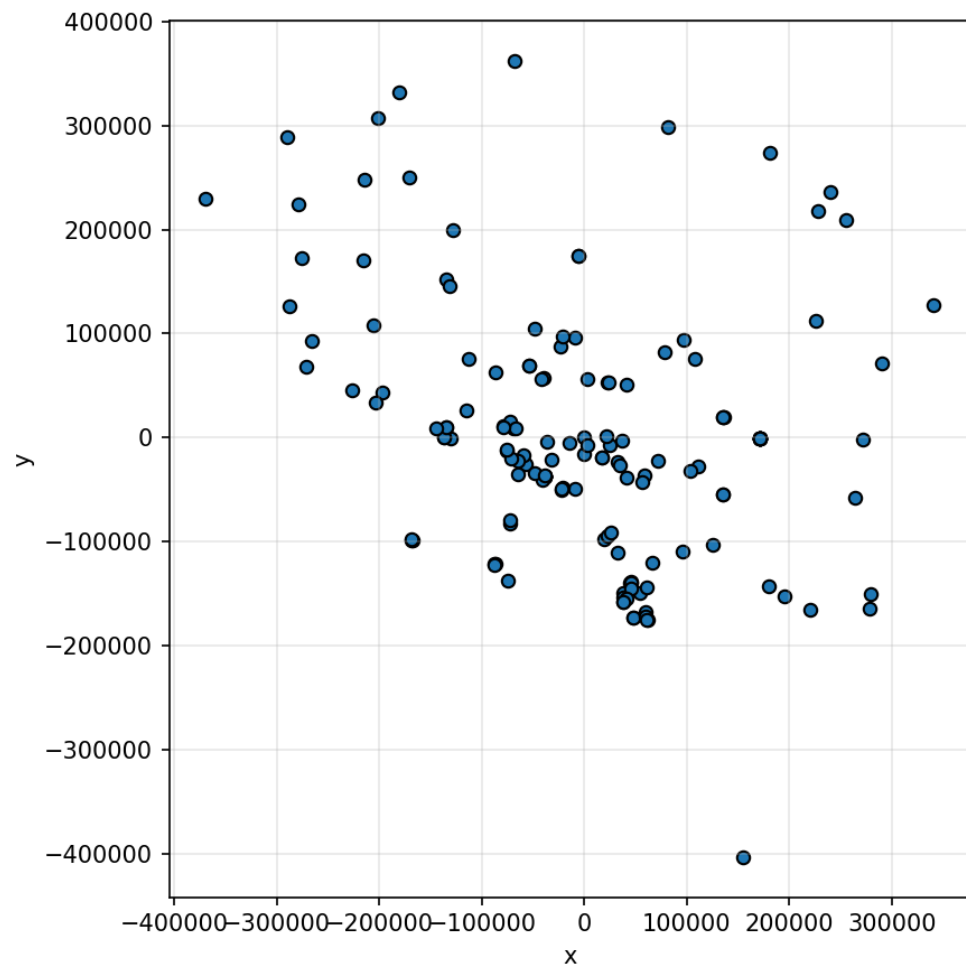
BOTTOM_LEFT_Y = -400_000

CELL_SIZE = 50_000

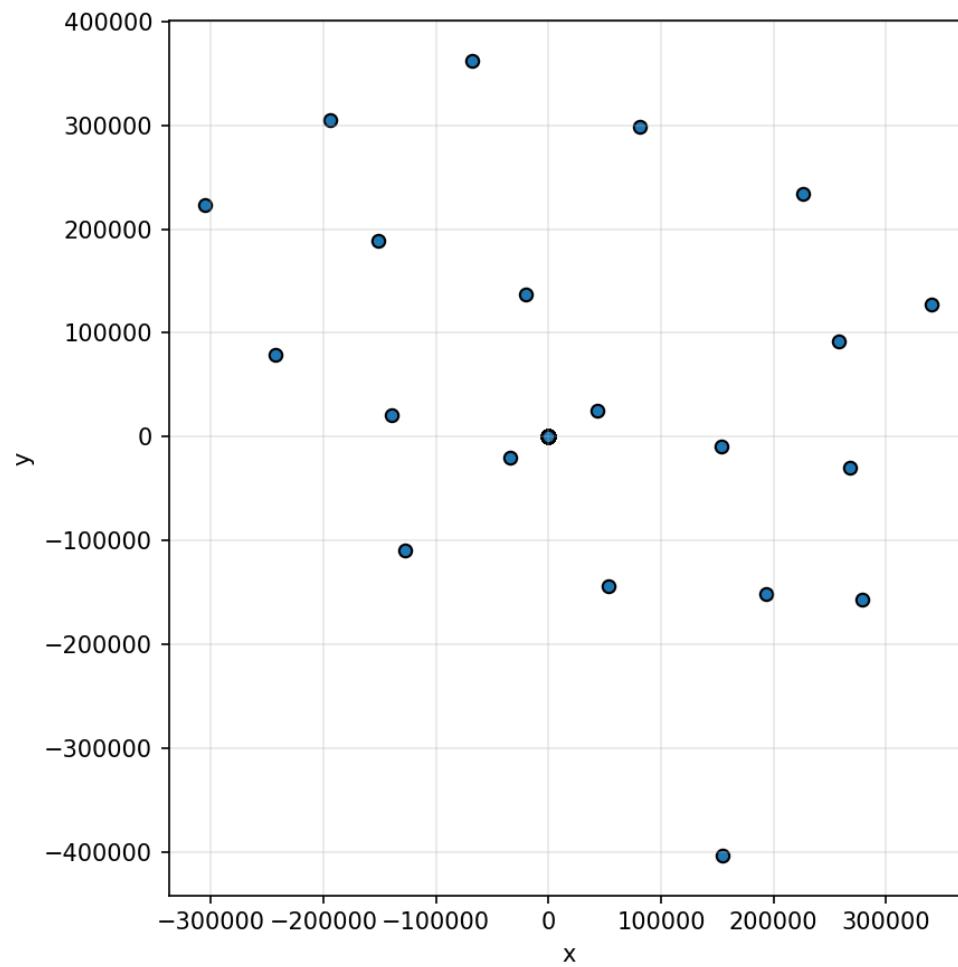
POINT_SIZE = 50_000

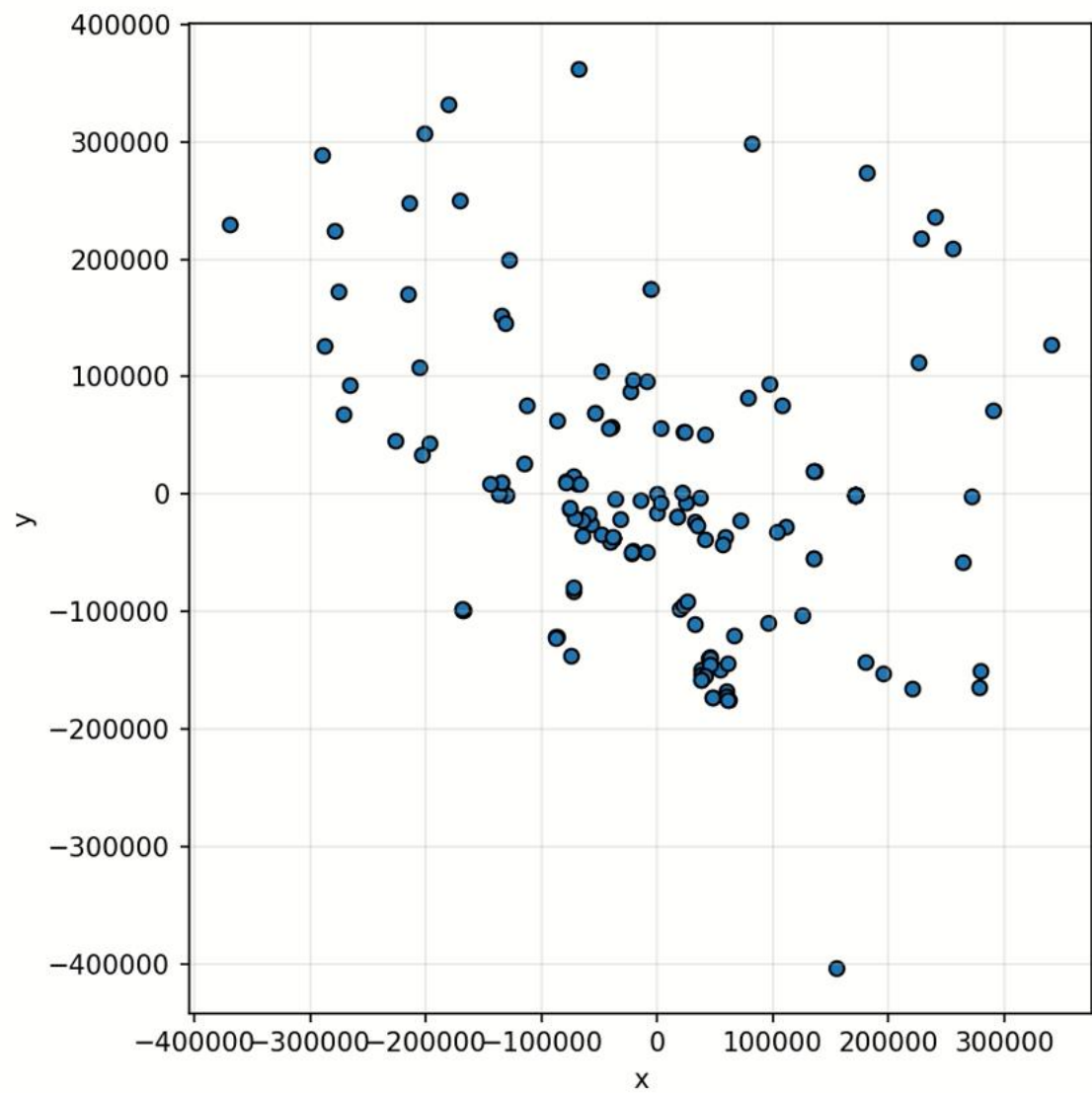
(large cell_size and point_size for emphasis of compression)

Before Compression



After Compression







Thank You